# DMA signal engine: timing, analog reading, and data age

**Technology Stack:** ESP32, ESP-IDF, I2S1 parallel DMA, ADC continuous driver, FreeRTOS

**Key Concepts:** Rendered bitstream, trigger/tail EOF, self-loop, seqlock snapshot, age gate, latency vs age

**Domain:** Embedded power-converter control

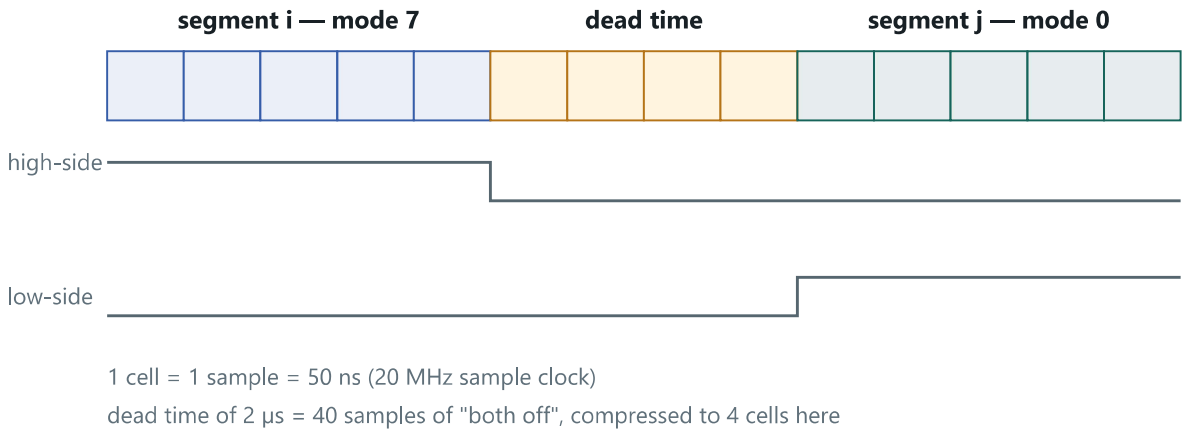
**Core Question:** When is the analog measurement taken relative to the signal, and can a stale value ever reach the controller?

## Document Index



## 1. Signal generation: timing from hardware, not code

The DMA engine ( `main/src/signal_engine_dma.cpp` ) does not toggle GPIOs from software. The whole signal cycle is pre-computed as a byte stream by `signal_render_bitstream()` and the I2S1 peripheral clocks it out in 8-bit parallel mode at **20 MHz — one byte every 50 ns**. Each byte carries the state of all six gate pins, one bit per lane. Dead time is not a runtime delay: it is 40 samples of "both switches off" baked into the stream (2 µs at the default dead time). Once the buffer plays, signal timing needs zero CPU involvement — jitter is bounded by the sample clock, not by interrupts.

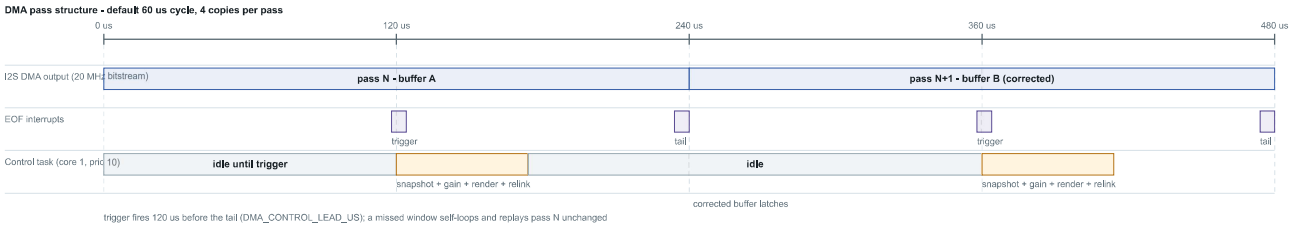



The rendered buffer is wrapped in a DMA descriptor chain ( `chain_build()` ) with two special EOF descriptors:

- **Trigger EOF** — placed `DMA_CONTROL_LEAD_US = 120 µs` of playback before the pass boundary. Its interrupt wakes the control task (Core 1, priority 10).
- **Tail EOF** — at the pass boundary. It counts passes and, by default, *links back to its own chain head* (self-loop).

The self-loop is the key safety property: if the control task does not finish inside the 120 µs window, the hardware simply replays the same pass. That is a *Missed Control Update* — the correction is skipped, but the power signal timing is completely untouched. When the control task does finish, it re-renders the inactive buffer with new corrections and swaps one pointer (the tail descriptor's `next` ), so the corrected buffer latches cleanly at the next pass boundary and the waveform can never tear.

Short cycles are copied back-to-back until a pass reaches at least 200 µs, so the control task is not woken at unsustainable rates: the default 60 µs cycle is rendered as 4 copies, giving a 240 µs pass.

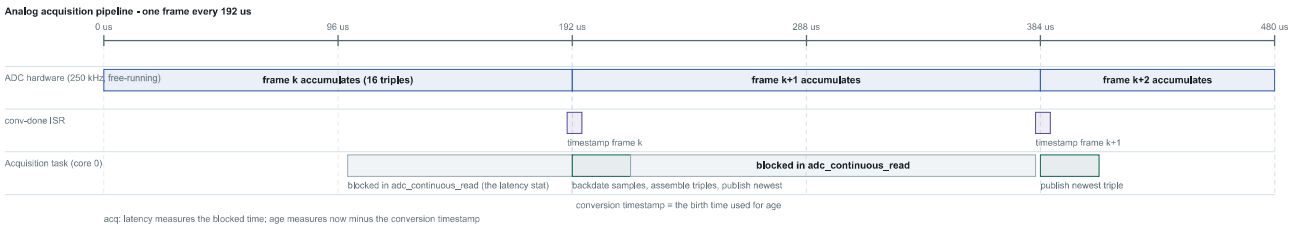



## 2. Reading the analog signals

The reading side ( `main/src/helper_analog.cpp` ) runs on Core 0, fully decoupled from the signal engine:

1. The ADC samples AN3/AN5/AN6 round-robin at 250 kHz aggregate — one triple every 12 µs, free-running, with *no phase relationship* to the signal edges.
2. Samples accumulate in the ADC DMA driver into frames of 16 triples (48 samples = **192 µs of signal time per frame**). The driver store holds up to 4 frames; `flush_pool = 1` prefers dropping old data over keeping a backlog.
3. When a frame completes, the `on_conv_done` ISR captures a timestamp — this is what makes the age of each sample measurable later.
4. The acquisition task reads the frame, **backdates every sample** to its true conversion time (frame timestamp minus position × sample period), assembles complete triples, and publishes **only the newest one** through a seqlock. Older triples in the same batch are discarded on purpose — the control point only ever wants "the latest".

Because the publisher writes a single snapshot behind a sequence counter, the Core-1 reader can always grab a coherent triple without locks, and a preempted Core-0 writer can never stall the control point: the seqlock read is bounded and gives up as a recoverable miss.




## 3. The age of an element vs the latency statistics

The status values `ADC Latency min/avg/max` and the snapshot `age` are **two different measurements taken at two different places —** and only age ever touches the values the controller uses.

The latency statistic is a stopwatch around the `adc_continuous_read()` call:

▶ Show code: cpp

It measures **how long the acquisition task sat blocked waiting for a frame —** a Core-0 health metric that is never attached to a sample and never gates a value:

| Statistic | Interpretation |
|---|---|
| min 3 µs | A completed frame was already waiting in the driver store; the call returned immediately. |
| avg 119 µs | The task usually arrives mid-frame and waits the remainder of the 192 µs accumulation (about half a frame ≈ 96 µs plus wakeup overhead). This is the *healthy* signature. |
| max 4481 µs | One call took 4.5 ms of wall clock. Frames complete every 192 µs, so the *task itself did not get CPU time* (Core 0 busy, typically a BLE burst) — the ADC never stopped. |

Age is computed at the moment of *use*, in `analog_read_control_snapshot()`, from the ISR-captured, backdated **conversion time**:

▶ Show code: cpp

The budget is max(signal cycle window, 288 µs pipeline floor). A rejected snapshot is a recorded miss and the DMA engine simply self-loops the previous buffer.

**Q: Am I using a value with the max latency?**
No — twice over. First, latency is not a property of any value; it is the reader task's blocking time. Second, even when a 4.5 ms stall makes the eventually-published data old, the age gate rejects it at the control point. The consequence of a stall is *missed correction updates, never actuation on old data*. One real side effect: 3 consecutive misses auto-disable the controller in `signal_control_update_corrections()` — if control has ever dropped to OFF by itself, these max-latency events are the likely cause.

**Q: Can some newer element be used instead?**
There is no newer *published* element to pick — the pipeline already discards everything except the newest triple per read. During a stall, newer samples do exist, but as raw unread bytes inside the ADC DMA store. The remedy is making the reader run sooner (Core-0 scheduling, smaller frames), not selecting a different element.

## 4. How generation and reading work together

The contract: a measurement taken during pass N is consumed at the trigger (120 µs before N ends), corrections are rendered and relinked before the tail, and pass N+1 plays them. The control cadence is one update per pass (240 µs for the default pattern), which sits comfortably above the 192 µs ADC frame period — so the DMA engine almost always finds a fresh snapshot waiting. (The CPU engine, by contrast, asks every ~60 µs and makes stale reads structural.)

| Quantity | Value | Source |
|---|---|---|
| Bitstream sample clock | 20 MHz (50 ns/sample) | `SIGNAL_DMA_SAMPLE_RATE_HZ` (D2) |
| Default cycle | 60 µs (10/20/10/20 µs, modes 7/0/7/0) | `signal_init_default_dataset()` |
| Pass floor / default pass | ≥ 200 µs → 4 copies = 240 µs | `DMA_MIN_PASS_SAMPLES` |
| Control lead | 120 µs before the tail | `DMA_CONTROL_LEAD_US` (D3/D4) |
| Dead time | 2 µs = 40 samples | `DEFAULT_DEAD_TIME_US` |
| ADC rate / triple pitch | 250 kHz aggregate / 12 µs per triple | `g_analog_continuous_sample_hz` |
| ADC frame | 16 triples = 192 µs; store 4 frames deep | `ADC_CONTINUOUS_FRAME_TRIPLES` |
| Age budget at control point | max(cycle window, 288 µs floor) | `analog_min_snapshot_age_us()` |
| Control auto-off threshold | 3 consecutive misses | `signal_control_update_corrections()` |

Source files: `main/src/signal_engine_dma.cpp` (renderer, descriptor chains, ISR, control task), `main/src/signal_controller.cpp` (corrections, age budget, auto-off), `main/src/helper_analog.cpp` (continuous pipeline, ISR timestamps, seqlock, age gate,

latency stopwatch), and `docs/adr/0001-dual-signal-engine-i2s-dma.md` (design decisions D1–D10).